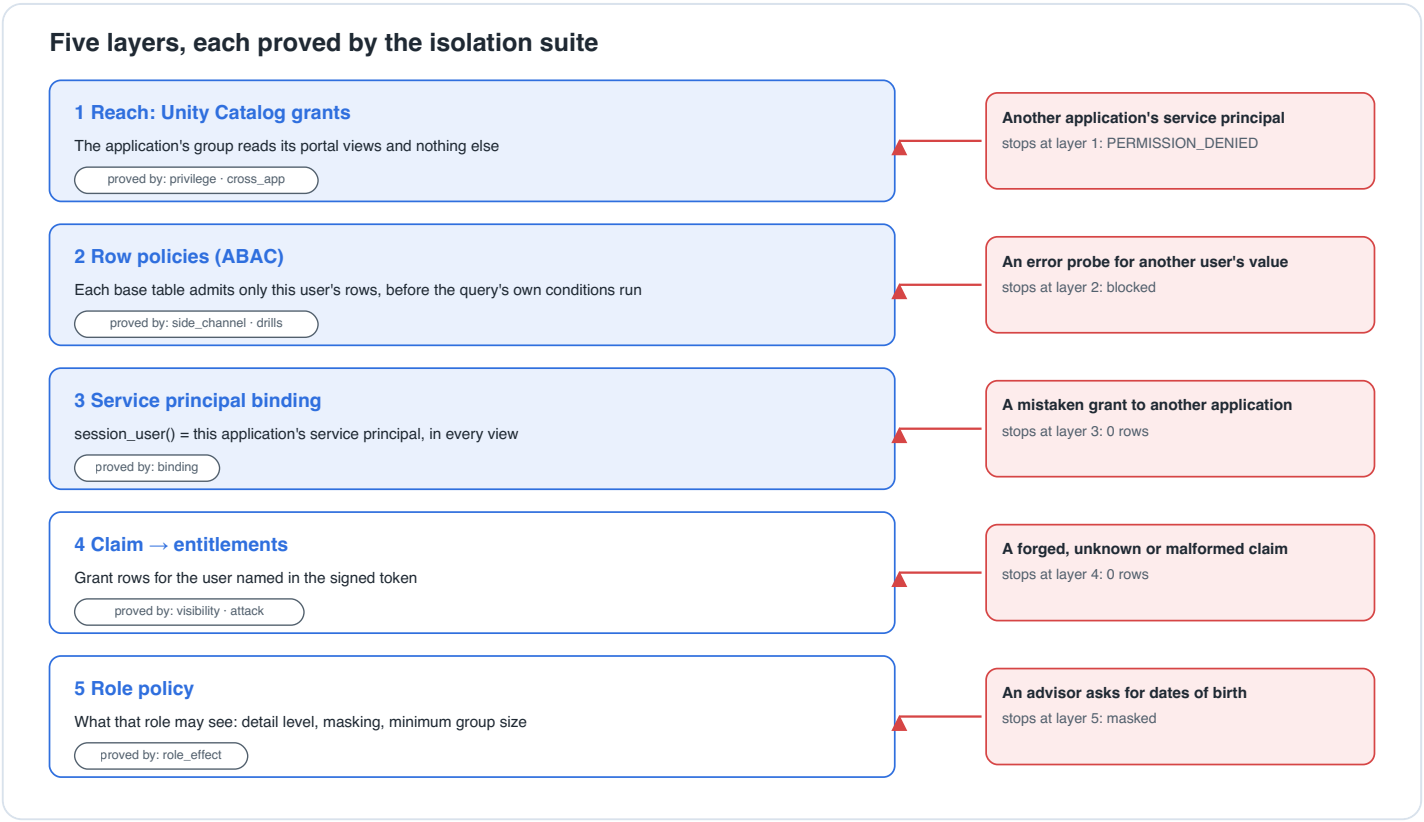
GENIE FOR EXTERNAL USERS · OAUTH CUSTOM IDENTITY CLAIMS

Admin Runbook

Community reference, not official Databricks documentation. OAuth custom identity claims are announced on the Databricks *What's coming?* page; check your workspace with `sql/check_availability.sql`.

This runbook is for the people who operate the deployment: platform administrators, data stewards and the team that owns the portal backend. It covers both enforcement models, the layered default and views alone. Choosing an Enforcement Model explains when to use which.



What each layer stops. Layers 1 and 2 are the ones this runbook operates.

Who needs what

Task	Needs
Create the two governed tags	An account admin, or a principal with permission to create governed tags

Task	Needs
Create service principals and account groups (bootstrap)	A workspace admin
Create row filter policies	`MANAGE` on the base schema, or ownership of it
Create the claim-keyed views and policy functions	`CREATE TABLE` and `CREATE FUNCTION` on the governance and portal schemas
Run portal queries	Serverless compute, a SQL warehouse, or Databricks Runtime 16.4+ for the principals the policies apply to

---

1. Create the governed tags

The row policies find their tables and columns through two governed tags. They're defined in `config/tags.yml`:

Tag	Set on	Allowed values
`portal_grain`	tables	`accounts`, `contributions`, `roster`, `tenant`, `reference`, `members`, `claims`, `accumulators`, `cells`, `plan_totals`, `roster_totals`, `remittance`
`portal_key`	columns	`member`, `tenant`, `pharmacy`, `cell_count`, `birth_date`

An account admin creates them once per account, in the UI or with the API:

```
databricks tag-policies create-tag-policy portal_grain --json '{"tag_key": "portal_grain",
  "values": [{"name": "accounts"}, {"name": "contributions"}, {"name": "roster"}, {"name":
"tenant"},
             {"name": "reference"}, {"name": "members"}, {"name": "claims"}, {"name":
"accumulators"}, {"name": "cells"},
             {"name": "plan_totals"}, {"name": "roster_totals"}, {"name": "remittance"}]}'
databricks tag-policies create-tag-policy portal_key --json '{"tag_key": "portal_key",
  "values": [{"name": "member"}, {"name": "tenant"}, {"name": "pharmacy"}, {"name": "cell_count"},
{"name": "birth_date"}]}'
```

The bootstrap job tries this for you with `tag_set=governed` and prints these instructions when it isn't allowed. Then give `ASSIGN` on both tags to a small governance group only (manage tag policy permissions). Tagging is a security boundary: whoever can change these tags can change which rows the portal sees.

The row policies task refuses to run when either tag isn't governed or lacks a value. A free-form tag with the same name would leave the policies matching nothing.

Evaluation workspaces: the stand-in set

If you can't create governed tags yet, deploy with `--var tag_set=stand_in`. It maps each logical tag to a platform system tag. Those tags aren't reserved for access control, so never use this set in production.

---

2. Groups and service principal rotation

Bootstrap creates one account group per application (`genie-claims-ref-<example>-portal-backends`) with the application's service principal as its only member. Grants and row policies name the group, never the service principal.

Grants and policies never need to change when the service principal does. The views and policy functions do: their `session_user()` check names the application's service principal. To replace it:

1. Create the new service principal and its credential, and store them in the secret scope under the example's keys.
2. Add it to the group, and wait until it can read a portal view. New memberships can take a few minutes to reach Unity Catalog.
3. Rerun the deploy job. Tasks 04 and 05 recreate the views and functions for the new service principal; from this point the old one gets no rows.
4. Switch the backend to the new credential, in the same change window as step 3.
5. Remove the old service principal from the group.

Rotating only the secret of the same service principal needs none of this: the application id stays the same.

---

3. Add a protected table

Order matters. Grant last.

1. Add the table to `row_policies` in the example's `example.yml`, with its grain and its key columns in function-argument order.
2. If it's a new grain, add a `rf_<grain>` function to `sql/30_policy_functions.sql`. A policy function may read only tables that carry no policy of their own. If the rule needs facts from a protected table, derive them into a governance table in `sql/05_derived_tables.sql`.
3. Deploy. Task 05 tags the table and its columns, creates the policy and checks coverage.
4. Add or change the claim-keyed view that reads it, then its grant in `sql/20_runtime_grants.sql`.
5. Run the isolation suite. A test in `tests/test_config.py` also fails offline if a portal view reads a base table that has no row policy.

If someone grants a new table before it's tagged, default-deny returns no rows to the portal until a steward tags it.

---

4. Change a policy function

Edit `sql/30_policy_functions.sql` and rerun the deploy job. Task 05 checks the tags first, replaces each function in place, then re-applies the policies and checks that every one is deployed. The policies stay attached throughout, so no table is ever unfiltered for the portal. In workspaces that raise without a claim, the service principal does the replacing: it gets `MANAGE` on each existing function and governance-only rights for the duration, and loses both again.

---

5. Guardrail queries for CI

Run these after every change to tags, policies, views or grants, and fail the build when one returns rows. Replace `<catalog>`, `<base>`, `<governance>` and `<group>`.

Base tables without a grain tag. These return nothing to the portal under default-deny, so a row here usually means a forgotten step:

```
SELECT t.table_name
FROM <catalog>.information_schema.tables AS t
LEFT JOIN <catalog>.information_schema.table_tags AS g
  ON g.schema_name = t.table_schema AND g.table_name = t.table_name AND g.tag_name =
'portal_grain'
WHERE t.table_schema = '<base>' AND t.table_type <> 'VIEW' AND g.tag_name IS NULL;
```

Key tags on more than one column of the same table, which make a policy ambiguous:

```
SELECT table_name, tag_value, count(*) AS columns
FROM <catalog>.information_schema.column_tags
WHERE schema_name = '<base>' AND tag_name = 'portal_key'
GROUP BY table_name, tag_value
HAVING count(*) > 1;
```

Grain-tagged tables (other than reference tables) with no key tag at all, so no row policy can bind to them:

```
SELECT g.table_name
FROM <catalog>.information_schema.table_tags AS g
LEFT JOIN <catalog>.information_schema.column_tags AS k
  ON k.schema_name = g.schema_name AND k.table_name = g.table_name AND k.tag_name = 'portal_key'
```

```
WHERE g.schema_name = '<base>' AND g.tag_name = 'portal_grain' AND g.tag_value <> 'reference'
GROUP BY g.table_name
HAVING count(k.column_name) = 0;
```

Any grant to the portal's group, or directly to its service principal, outside the portal schema, at table, schema or catalog level:

```
SELECT 'table' AS level, table_schema AS name, privilege_type FROM
<catalog>.information_schema.table_privileges
WHERE grantee IN ('<group>', '<service principal application id>') AND table_schema IN ('<base>',
'<governance>')
UNION ALL
SELECT 'schema', schema_name, privilege_type FROM <catalog>.information_schema.schema_privileges
WHERE grantee IN ('<group>', '<service principal application id>') AND schema_name IN ('<base>',
'<governance>')
UNION ALL
SELECT 'catalog', catalog_name, privilege_type FROM
<catalog>.information_schema.catalog_privileges
WHERE grantee IN ('<group>', '<service principal application id>') AND privilege_type NOT IN
('USE_CATALOG', 'USE CATALOG');
```

The policies in place:

```
SHOW POLICIES ON SCHEMA <catalog>.<base>;
```

Notebook 05 runs the same coverage check (`claims_kit.policies.coverage_problems`) and a check that every policy is deployed (`policy_problems`) on every deploy, and the isolation suite repeats the second one. Also alert on tag changes in the audit log system table for your catalog, made by anyone outside the governance group.

---

6. Run the misconfiguration drills

The `northwind_drills` and `larkspur_drills` jobs (or `python -m claims_kit.drills --example <name>`) make five mistakes one at a time: - untag a key column; - point a policy at another group; - grant a view without the claim filter; - expose a new untagged table; - grant a base table directly; - expose a masked column through a view without the claim filter, read by a persona whose role masks it.

Each drill measures what the portal returns with the mistake in place and with one layer alone, restores the original state, and verifies every layer: the portal view, the policy's reach, the tags and the deployed policies. The drills stop if a restore can't be verified, and don't start on a broken deployment. Run them on a test deployment, never one that serves real users.

7. Operate views-only mode

`--var enforcement=views` deploys the claim-keyed views without row policies. It's reasonable when your application controls every query. With free-form questions, users can learn facts about other users' rows through errors (see the probe results in the enforcement model guide). If you must run it for a while:

- return a generic message to users when a query fails, never the error text (this narrows the channel; answered or failed is still one bit);
- alert when the portal's service principal hits `DIVIDE_BY_ZERO`, `CAST_INVALID_INPUT` or similar errors in query history, which is what probing looks like;
- plan the move to layered: create the governed tags and redeploy with the default.

8. Troubleshooting

Symptom	Cause and fix
Task 05 says the governed tags aren't ready	Create `portal_grain` and `portal_key` with every value (step 1), or use `tag_set=stand_in` for evaluation
Task 05 reports coverage problems	A table lacks its grain tag, a key column lacks its tag, or a key tag sits on two columns. Fix the tags; never remove a tag to silence the check.
`UC_ABAC_MULTIPLE_ROW_FILTERS`	Two row policies match one table. Give each table exactly one `portal_grain` value.
`UNSUPPORTED_NESTED_ROW_OR_COLUMN_ACCESS_POLICY`	A policy function reads a table that has its own policy. Read a governance table instead (step 3).
`UC_USING_REF_OF_MASKED_COLUMNS`	A column-mask policy masks a column that a row filter on the same table also reads. Mask a different column, or give the role a precomputed copy (as Larkspur does for pharmacy staff).

Symptom	Cause and fix
`UC_DEPENDENCY_DOES_NOT_EXIST` on a protected table during task 04	A policy function still reads a governance view that task 04 is rebuilding, usually after you changed which views the functions read. Remove the example's policies once (deploy with `enforcement=views`), then redeploy layered.
`ROW_LEVEL_SECURITY_FEATURE_NOT_SUPPORTED.VIEWS`	A schema-level policy matched a view. Keep row policies on schemas that hold only tables (the base schema here).
`OAUTH_CUSTOM_IDENTITY_CLAIM_NOT_PROVIDED` creating a policy function	The workspace raises without a claim; the functions must be created from a claim session, which task 05 does
`INSUFFICIENT_PERMISSIONS` right after bootstrap	Group membership hasn't reached Unity Catalog yet. Wait a few minutes; the isolation suite waits up to five.
`POLICY_NOT_FOUND` on `DROP POLICY`	`DROP POLICY` has no `IF EXISTS`; the tooling treats this as already removed
Query history shows 0 bytes read for portal queries	Expected for principals that policies apply to. Tune performance with a principal outside the policy.